

Queue ADT and Array-based Queues

Algorithms and Data Structures
NEU 2026

Linear Structures

- Part 1

- Linear Structures
- Stack ADT and Array-based Stacks

- Part 2

- Queue ADT and Array-based Queues

- Part 3

- List and Iterator ADTs
- Amortisation

Queue ADT

- ❑ Stores arbitrary objects
- ❑ Insertions and deletions are first-in first-out (FIFO)
- ❑ Think of a waiting line (queue)
 - insertions at rear of queue
 - removals at front of queue



Queue Operations

- ❑ `enqueue(T)` – inserts an element at the end of the queue
- ❑ `T dequeue()` – removes and returns the element at the front of the queue
- ❑ `T first()` – returns the element at the front without removing it
- ❑ `integer size()` – returns the number of elements stored
- ❑ `boolean isEmpty()` – indicates whether no elements are stored

Queue Boundary Case

- Attempting execution of dequeue or first on an empty queue
 - return null
 - throw NoSuchElementException

Example

<i>Operation</i>	<i>Return</i>	<i>Queue Contents</i>
enqueue(5)	—	(5)
enqueue(3)	—	(5, 3)
dequeue()	5	(3)
enqueue(7)	—	(3, 7)
dequeue()	3	(7)
first()	7	(7)
dequeue()	7	()
dequeue()	<i>null</i>	()
isEmpty()	<i>true</i>	()
enqueue(9)	—	(9)
enqueue(7)	—	(9, 7)
size()	2	(9, 7)
enqueue(3)	—	(9, 7, 3)
enqueue(5)	—	(9, 7, 3, 5)
dequeue()	9	(7, 3, 5)

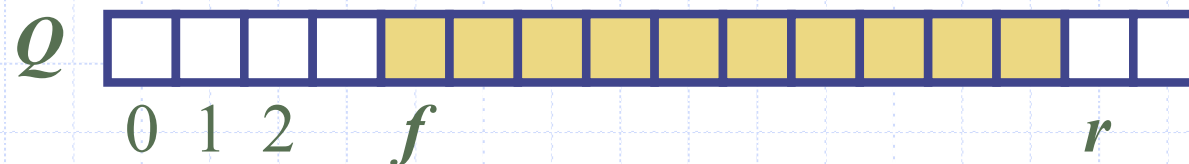
Applications of Queues

- ❑ Direct applications
 - Waiting lists, bureaucracy
 - Access to shared resources (e.g. printer)
 - Process / Thread scheduling
 - Multi-programming / time-sharing
- ❑ Indirect applications
 - Auxiliary data structure for algorithms
 - Component of other data structures

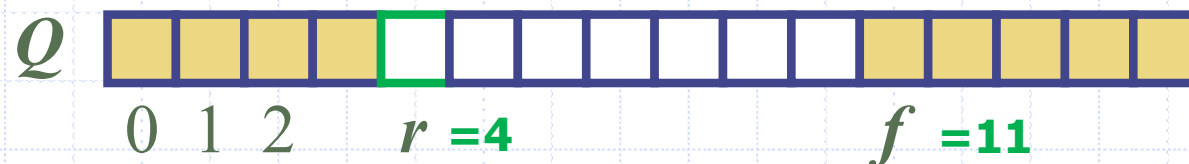
Array-based Queue

- Use an array of length N in a circular fashion
- Two variables keep track of the front and size
 - f index of the front element
 - s number of stored elements
- When the queue has fewer than N elements, array location $r = (f + s) \bmod N$ is the first empty slot past the rear of the queue

normal configuration



wrapped-around configuration

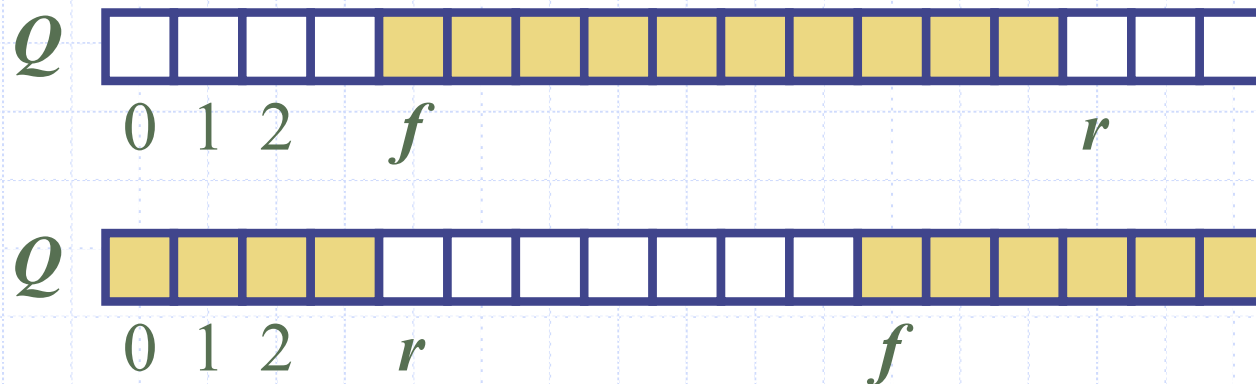


$s=9, N=16$

Queue Operations

Algorithm *size()*
return *sz*

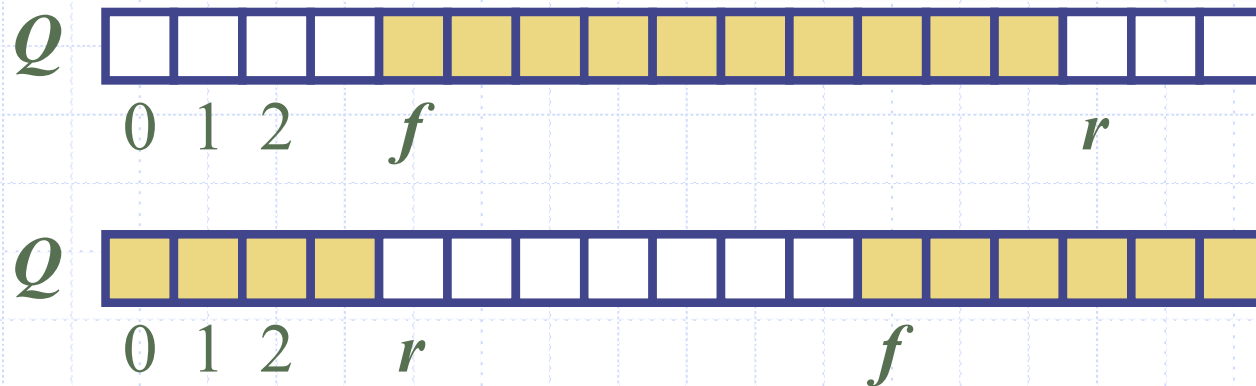
Algorithm *isEmpty()*
return (*sz* == 0)



Queue Operations (cont.)

- ❑ enqueue throws an exception if array is full
 - implementation-dependent
- ❑ Uses modulo operator
 - remainder of division

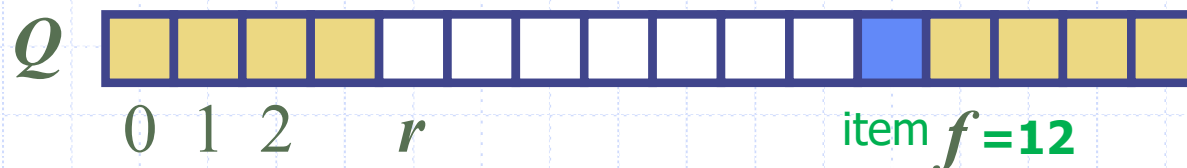
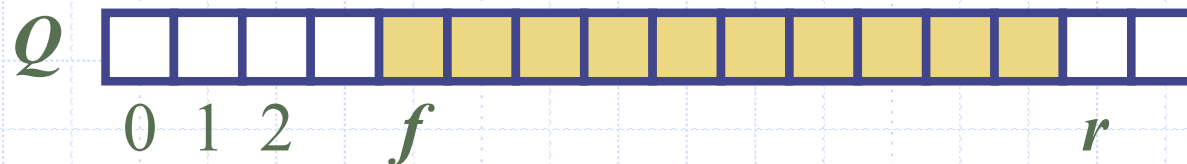
```
Algorithm enqueue(item)  
  if size() = N then  
    throw IllegalStateException  
  else  
     $r \leftarrow (f + sz) \bmod N$   
     $Q[r] \leftarrow item$   
     $sz \leftarrow (sz + 1)$ 
```



Queue Operations (cont.)

- Note that dequeue returns null if the queue is empty

```
Algorithm dequeue()  
  if isEmpty() then  
    return null  
  else  
    item  $\leftarrow Q[f]$   
     $f \leftarrow (f + 1) \bmod N$   
     $sz \leftarrow (sz - 1)$   
    return item
```



$sz = 9, N = 16$

Queue Interface in Java

- Java interface corresponding to our Queue ADT
- Assumes `first()` and `dequeue()` return null if queue is empty

```
public interface Queue<T> {  
    int size();  
    boolean isEmpty();  
    T first();  
    void enqueue(T element);  
    T dequeue();  
}
```

Comparison to java.util.Queue

- Our Queue methods and corresponding methods of **java.util.Queue**

Our Queue ADT	Interface java.util.Queue	
	throws exceptions	returns special value
enqueue(<i>e</i>)	add(<i>e</i>)	offer(<i>e</i>)
dequeue()	remove()	poll()
first()	element()	peek()
size()	size()	
isEmpty()	isEmpty()	

Array-based Implementation

```
1  /** Implementation of the queue ADT using a fixed-length array. */
2  public class ArrayQueue<E> implements Queue<E> {
3      // instance variables
4      private E[ ] data;                // generic array used for storage
5      private int f = 0;                // index of the front element
6      private int sz = 0;                // current number of elements
7
8      // constructors
9      public ArrayQueue() {this(CAPACITY);} // constructs queue with default capacity
10     public ArrayQueue(int capacity) {      // constructs queue with given capacity
11         data = (E[ ]) new Object[capacity]; // safe cast; compiler may give warning
12     }
13
14     // methods
15     /** Returns the number of elements in the queue. */
16     public int size() { return sz; }
17
18     /** Tests whether the queue is empty. */
19     public boolean isEmpty() { return (sz == 0); }
20 }
```

Predefined constant



Array-based Implementation (2)

```
21  /** Inserts an element at the rear of the queue. */
22  public void enqueue(E e) throws IllegalStateException {
23      if (sz == data.length) throw new IllegalStateException("Queue is full");
24      int avail = (f + sz) % data.length;    // use modular arithmetic
25      data[avail] = e;
26      sz++;
27  }
28
29  /** Returns, but does not remove, the first element of the queue (null if empty). */
30  public E first() {
31      if (isEmpty()) return null;
32      return data[f];
33  }
34
35  /** Removes and returns the first element of the queue (null if empty). */
36  public E dequeue() {
37      if (isEmpty()) return null;
38      E answer = data[f];
39      data[f] = null;    // dereference to help garbage collection
40      f = (f + 1) % data.length;
41      sz--;
42      return answer;
43  }
```

Why set to null?

Array-based Implementation (3)

```
/** Removes and returns the first element of the queue (null if empty). */  
public E dequeue() {  
    if (isEmpty()) return null;  
    E answer = data[f];  
    data[f] = null; // dereference to help garbage collection  
    f = (f + 1) % data.length;  
    sz--;  
    return answer;  
}
```

Why set to null?

- **Why Set to Null?:** By setting `data[f]` to null, you remove the reference to the object that was stored there. In Java, objects are only eligible for garbage collection when there are no more references pointing to them. If you didn't set the array element to null, the reference would remain, and the garbage collector wouldn't be able to reclaim that memory, even though you no longer need the object.

Performance and Limitations

□ Performance

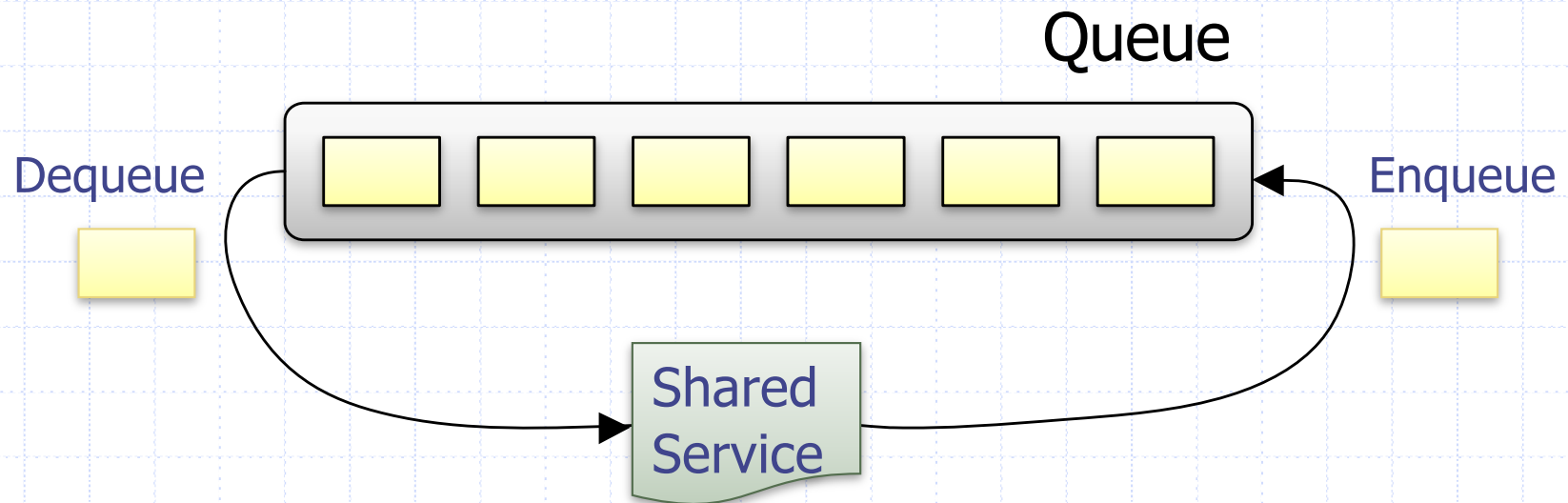
- Let n be the number of elements in the queue
- Let N be the maximum size of the queue
- Space used is $O(N)$
- Each operation runs in time $O(1)$

□ Limitations

- Maximum size of the queue must be defined *a priori* and cannot be changed
- Pushing a new element into a full queue causes an implementation-specific exception

Application: Round Robin Scheduler

- Can be implemented using a queue Q by repeatedly performing the following steps:
 - $e = Q.dequeue()$
 - Service element e
 - $Q.enqueue(e)$



Application: Round Robin Scheduler (2)

How the Queue is Used:

In this example, the queue is used to manage the tasks or processes that need to be scheduled:

1. Dequeue a Task:

- The scheduler dequeues (removes) the first task from the front of the queue.
- `e = Q.dequeue()`: This step retrieves the first task `e` from the queue `Q`.

2. Service the Task:

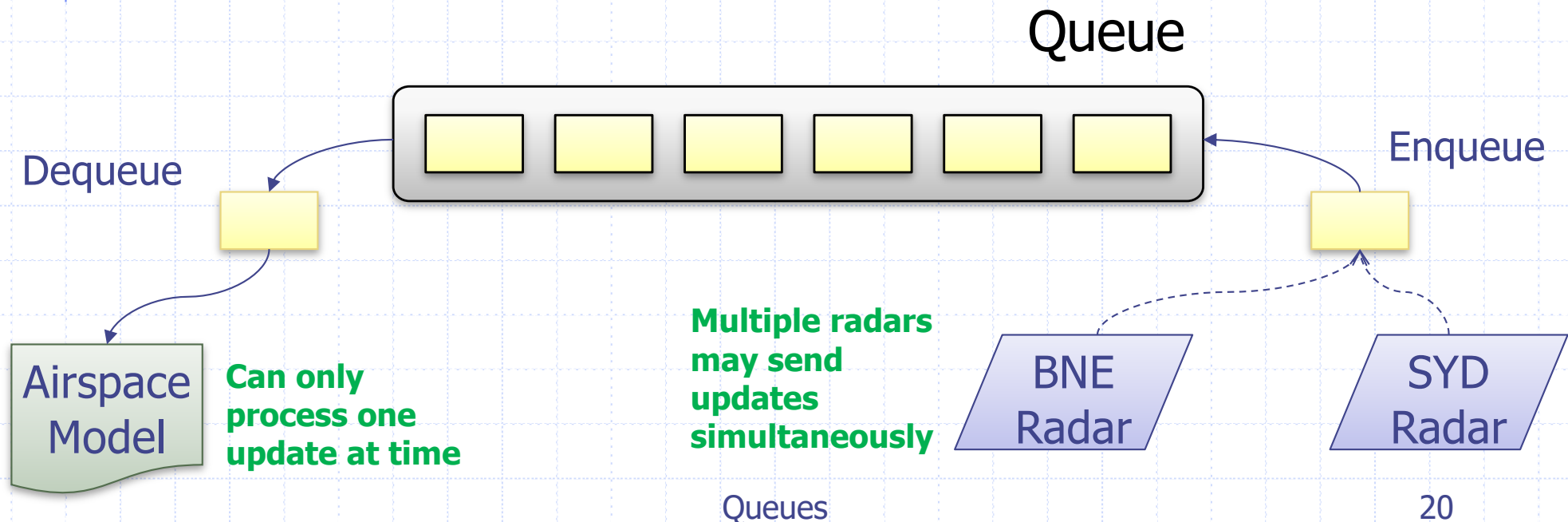
- The task `e` is then serviced or executed for a certain amount of time (the "time slice").
- This step represents the actual processing of the task.

3. Enqueue the Task:

- After the task has been serviced for its allotted time, it is enqueued (added) back to the end of the queue.
- `Q.enqueue(e)`: This step puts the task `e` back at the end of the queue, so it will eventually be serviced again after all other tasks have had their turn.

Application: Shared Resource

- ❑ Multiple sensors updating a single data repository
 - e.g. multiple radar sites updating airspace model
- ❑ Sensors don't worry about conflicting actions on model
 - queue needs to be thread safe



Summary

- Queue ADT
 - array implementation
 - ◆ performance & limitations
- Queue applications